

## Components used:

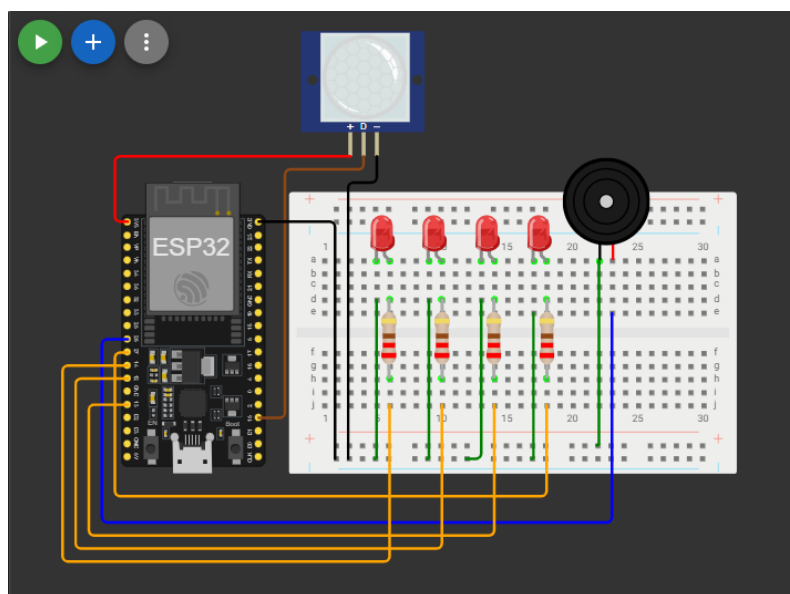
- ESP32 Microcontroller
- PIR motion sensor
- LEDs
- Alarm buzzer
- Resistors
- Network

## Working Principle:

The core automation upgrade focused on correcting the inherent instability of PIR sensors, which often send intermittent LOW signals during continuous motion (known as "chatter").

1. **Input Reading:** The system constantly reads the state of **GPIO 15** in the loop() function.
2. **Alarm Activation:** If currentPIRState is HIGH, the boolean flag isAlarmActive is set to true instantly, and the alarm sequence begins.
3. **Cooldown Timer (Automation Fix):** If the PIR state transitions from HIGH to LOW (motion stops), the system does **not** immediately turn off the alarm. Instead, it starts the motionStoppedTime timer.
  - The alarm remains active (isAlarmActive = true) for a minimum of **2.5 seconds** (kPIR\_RESET\_TIME\_MS).
  - This ensures that the alarm doesn't abruptly stop if the intruder momentarily steps out of the sensor's field of view or if the sensor produces a brief low reading, significantly enhancing reliability.

## Circuit Diagram:



## Code:

```
// =====  
// ESP32 PIR Motion Alarm System  
// 1 PIR input, 4 Blinking LEDs, 1 Tone Buzzer  
// =====  
  
// --- INPUT PINS ---  
const int kPirPin = 15;  
  
// --- OUTPUT PINS ---  
const int kBuzzerPin = 26; // Buzzer pin  
const int kLed1Pin = 13;   // LED 1 pin  
const int kLed2Pin = 12;   // LED 2 pin  
const int kLed3Pin = 14;   // LED 3 pin  
const int kLed4Pin = 27;   // LED 4 pin  
  
// --- ALARM TIMING & TONES ---  
const unsigned long kBlinkIntervalMs = 150; // LED blink frequency  
const int kToneFrequency = 2500;           // Buzzer tone frequency (1000 Hz)  
  
// Time the alarm must remain active after PIR signal goes LOW.  
// This prevents quick on/off cycling caused by PIR sensor timing.  
const unsigned long kPIR_RESET_TIME_MS = 2500; // 2.5 seconds  
  
// --- STATE VARIABLES ---  
bool isAlarmActive = false;  
unsigned long lastBlinkTime = 0;  
unsigned long motionStoppedTime = 0;  
bool isLedsOn = false; // Controls the blinking state of the LEDs  
  
// Function to handle the alarm outputs  
void handleAlarmOutputs() {  
    if (isAlarmActive) {  
        // 1. BUZZER: Ring using tone() for proper oscillation  
        tone(kBuzzerPin, kToneFrequency);  
  
        // 2. LEDS: Blink using non-blocking timer  
        if (millis() - lastBlinkTime >= kBlinkIntervalMs) {  
            isLedsOn = !isLedsOn; // Toggle the state (ON -> OFF or OFF -> ON)  
            lastBlinkTime = millis();  
        }  
  
        // Apply the current blinking state to all 4 LEDs  
        int ledState = isLedsOn ? HIGH : LOW;  
        digitalWrite(kLed1Pin, ledState);  
        digitalWrite(kLed2Pin, ledState);  
        digitalWrite(kLed3Pin, ledState);  
        digitalWrite(kLed4Pin, ledState);  
    }  
}
```

```

    digitalWrite(kLed4Pin, ledState);

} else {
    // Ensure ALL outputs are OFF when no alarm is active
    noTone(kBuzzerPin); // Stop the buzzer tone
    digitalWrite(kLed1Pin, LOW);
    digitalWrite(kLed2Pin, LOW);
    digitalWrite(kLed3Pin, LOW);
    digitalWrite(kLed4Pin, LOW);
    isLedsOn = false; // Reset blinking state
}
}

void setup() {
    Serial.begin(115200);

    // Configure Input Pin
    pinMode(kPirPin, INPUT);

    // Configure Output Pins (Initialize OFF)
    pinMode(kBuzzerPin, OUTPUT);
    pinMode(kLed1Pin, OUTPUT);
    pinMode(kLed2Pin, OUTPUT);
    pinMode(kLed3Pin, OUTPUT);
    pinMode(kLed4Pin, OUTPUT);

    // Ensure all outputs start LOW
    digitalWrite(kLed1Pin, LOW);
    digitalWrite(kLed2Pin, LOW);
    digitalWrite(kLed3Pin, LOW);
    digitalWrite(kLed4Pin, LOW);

    Serial.println("-----");
    Serial.println("PIR Alarm System Initialized (4 LEDs + Buzzer Tone).");
    Serial.println("Monitoring PIR on GPIO 15.");
    Serial.println("-----");
}

void loop() {

    // 1. --- READ PIR STATE ---
    bool currentPIRState = (digitalRead(kPirPin) == HIGH);

    if (currentPIRState) {
        // If motion is detected, immediately activate the alarm and clear the stop
timer
        if (!isAlarmActive) {
            Serial.println("\n*** ALARM TRIGGERED: MOTION DETECTED ***");
            isAlarmActive = true;
        }
    }
}

```

```

    motionStoppedTime = 0; // Clear the timer if motion is active

} else {
    // Motion stopped (PIR is LOW)

    if (isAlarmActive && motionStoppedTime == 0) {
        // Start the cooldown timer only when the alarm is currently active and the
        timer hasn't started
        motionStoppedTime = millis();
    }

    // Check if the alarm cooldown period has expired
    if (motionStoppedTime != 0 && millis() - motionStoppedTime >=
kPIR_RESET_TIME_MS) {
        if (isAlarmActive) {
            // Cooldown expired, shut down the alarm
            Serial.println("--- Alarm Reset: Cooldown Expired ---");
            isAlarmActive = false;
        }
        motionStoppedTime = 0; // Reset the timer
    }
}
handleAlarmOutputs();
}

```

## Output:

